Electric Board Maps

Compact octilinear drawings of gendered forests,
from one circuit board to a million vertices

Abstract

Korst, Pronk and van Wijk's *family metro map* draws family relations in the octilinear style of a metro diagram and encodes gender in the orientation of the segment at a person: orthogonal for women, diagonal for men. The *Electric Board Map* transfers this idea to a printed circuit board. Vertices of a forest become *pads* on integer grid points, edges become *traces* of unit octilinear steps, and the direction of the last step into a vertex encodes its gender. The goal is a board that wastes as little area as possible. On one board, large trees are routed generation by generation with straight, one-bend, dogleg and chamfered-corner traces, chosen by a cost of five terms—length, bends, growth of the board, parallel "bus" runs and one look-ahead term; exact branch and bound, a maze router, rip-up and a squeeze pass complete the routing; conflicts that remain are repaired locally. The forests of interest consist of large trees (≥ 9 vertices) and small trees (≤ 3 vertices) holding about half of the vertices each. Trees of at most three vertices are drawn from straight-only templates and nestled into the gaps of the large ones. For forests of up to 10^6 vertices, the forest is divided at random into sub-forests of any chosen number of vertices; dividing by vertices gives boards of nearly equal size, whose loads provably differ by at most one tree. Every sub-forest is routed on its own board in parallel, the equal boards tile a grid with little wasted space, and the grid is drawn with a level-of-detail renderer. We describe both algorithms completely, prove their main properties, and report measurements from 100 to 1 000 000 vertices.

Contents

1 Introduction **1**

2 Drawing model **3**

3 Input and validation **4**

4 Routing a large tree **5**

5 Forest packing **11**

6 Verification **13**

7 Forests of up to a million vertices **14**

8 Running time **18**

9 Results **19**

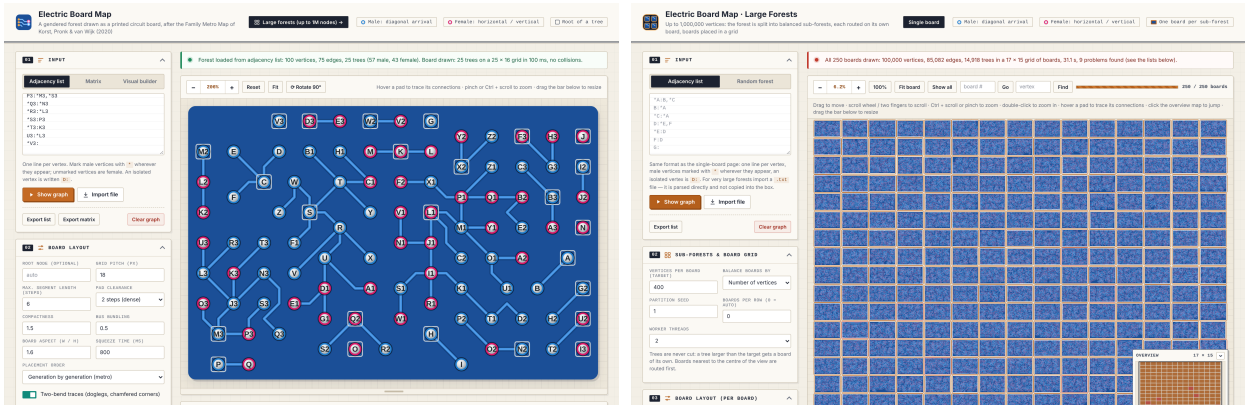
10 Discussion and limitations **22**

A Parameters **23**

1 Introduction

The family metro map [1] draws a family as a metro line whose end stations are the parents and whose intermediate stations are the children. All segments are horizontal, vertical or diagonal, and the orientation of a station's segment encodes gender: women sit on horizontal or vertical segments, men on diagonal ones. The result is a compact and attractive overview of family relations.

The *Electric Board Map* keeps three ingredients of that design: the **octilinear** style, the **gender rule** (*orthogonal = female, diagonal = male*) and a **generation-by-generation** construction. It replaces the



(a) one board (100 vertices)

(b) many boards (100 000 vertices on 250 boards)

Figure 1: The two scales of the algorithm, as shown by an interactive viewer (screenshots).

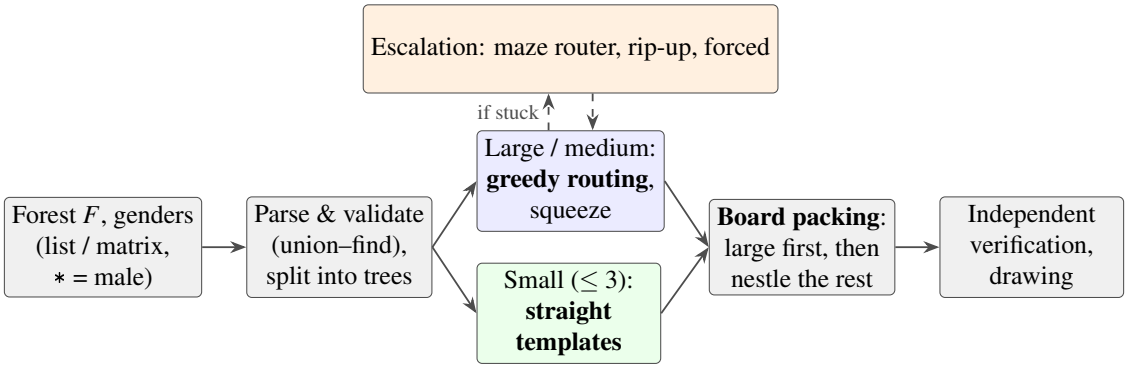


Figure 2: Pipeline of the one-board algorithm. Large and medium trees are routed (blue); the escalation steps (orange) are used only when no simple trace fits; small trees skip the router (green). Every edit re-runs the pipeline.

metro line by a printed circuit board. Every vertex is a pad on an integer grid point, every parent–child edge an independent copper trace on the grid, and the gender of a vertex is read from the last step of the trace that arrives at it. Traces may bend with unequal segment lengths and may run on adjacent grid lines, so the board can be filled densely; the aim is to **use the board area as well as possible**.

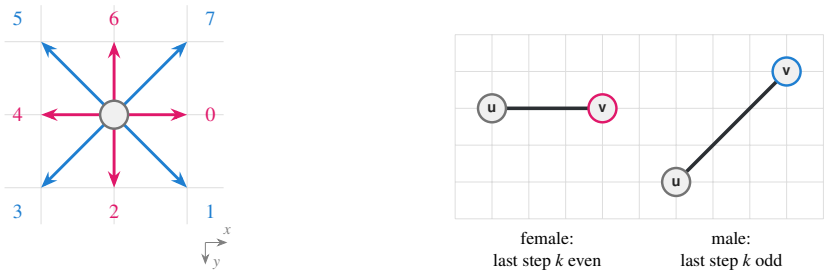
The forests. The algorithm is designed for one class of forests. Their trees are either *large*, with at least 9 vertices, or *small*, with at most 3 vertices (isolated vertices, single edges and paths of length two); *medium* trees of 4–8 vertices are rare. About half of the vertices lie in large trees and half in small trees, so a forest has a few dozen large trees for every few hundred small ones. Small trees are cheap and must be placed well; large trees are hard to route and set the shape of the board.

One board and many boards. The algorithm works at two scales (Figure 1):

- *One board* for a forest of up to about a thousand vertices (Sections 2–6): every tree is drawn on a single board, which is packed as densely as possible.
- *Many boards* for forests of up to 10^6 vertices (Section 7): the forest is divided into sub-forests of a chosen number of vertices, each sub-forest is drawn on its own board by the one-board algorithm, in parallel, and the boards are placed in a grid of which only the visible part is drawn.

Contributions.

- A precise drawing model with the gender rule as a constraint on the arrival direction, and proofs that valid drawings are crossing-free and that vertices of degree above eight cannot be drawn (Section 2).



(a) The 8 directions D_k (y down).

(b) Gender rule on arrival at the child v.

Figure 3: Directions and the gender rule. Pads are drawn with a magenta ring for female and a blue ring for male vertices (black: gender irrelevant in the illustration).

- A routing algorithm for one tree with four trace shapes, a cost of five terms (length, bends, compactness, bus and a single exit look-ahead that combines free directions and free landing points), an exact branch-and-bound pruning, escalation to a maze router and rip-up, and a squeeze compaction that provably never enlarges the board (Section 4).
- A packing of one board in which every tree of two vertices is a straight trace (proved), trees may be rotated and reflected (proved valid), and small trees are nestled into free cells (Section 5).
- A pipeline for a million vertices: linear-time reading, a random partition into sub-forests of any chosen number of vertices whose sizes differ by at most the largest tree (proved), parallel routing ordered by the user's view, a grid of nearly equal boards that wastes little space, and a level-of-detail renderer (Section 7).
- Measurements from 100 to 1 000 000 vertices (Section 9).

2 Drawing model

2.1 Input

The input is a forest $F = (V, E)$ and a gender function $g : V \to \{\mathrm{M}, \mathrm{F}\}$. One board is used for forests of up to about 1000 vertices; larger forests, up to 10^6 vertices, are divided into boards (Section 7). Each connected component is a tree. For a tree T with a chosen root r, every non-root vertex v has a parent $\mathrm{par}(v)$; $\mathrm{ch}(v)$ is the list of children, $\kappa(v) = |\mathrm{ch}(v)|$, and $s(v)$ is the number of vertices of the sub-tree rooted at v.

2.2 Grid, directions and traces

Drawings live on the integer grid $\mathbb{Z}^2$ with the y-axis pointing *down* (screen coordinates). The eight octilinear directions are numbered $k = 0, \ldots, 7$ clockwise on screen:

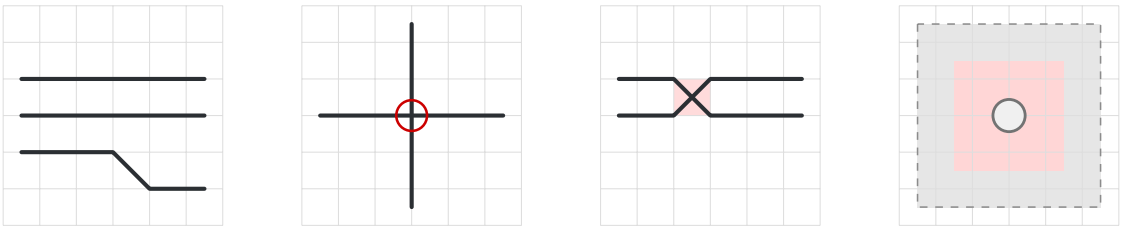
$$D_0 = (1,0), \quad D_1 = (1,1), \quad D_2 = (0,1), \quad D_3 = (-1,1),$$
$$D_4 = (-1,0), \quad D_5 = (-1,-1), \quad D_6 = (0,-1), \quad D_7 = (1,-1).$$

Direction k is *orthogonal* if k is even and *diagonal* if k is odd; turning by one index is a 45° bend. A unit step has length 1 (orthogonal) or $\sqrt{2}$ (diagonal); see Figure 3.

Definition 1 (Drawing)**.** *A drawing assigns to every vertex v a* pad $p(v) \in \mathbb{Z}^2$ *and to every edge $e = \{u, v\}$, with u the parent of v, a* trace $\pi(e) = (q_0, q_1, \ldots, q_n)$ *of grid points with $q_0 = p(u)$, $q_n = p(v)$ and $q_i - q_{i-1} \in \{D_0, \ldots, D_7\}$ for all i. The points $q_1, \ldots, q_{n-1}$ are the* interior points *of the trace and $q_n - q_{n-1}$ is its* arrival direction.

2.3 Constraints

Let $c \in \{2, 3\}$ be the *pad clearance*. A drawing is *valid* if



(a) Traces on adjacent grid lines: *allowed.* (b) Shared grid point: *forbidden* by (C2). (c) Diagonals crossing in one unit square: *forbidden* by (C3). (d) No other pad on the red points (c=2) or grey points (c=3).

Figure 4: Legality of traces and pads. Because traces may sit on neighbouring grid lines, parallel traces pack densely; only shared grid points, crossing diagonals and pads closer than the clearance are excluded.

(C1) **Pad clearance.** $\|p(v) - p(w)\|_\infty \geq c$ for all distinct v, w in the same tree (pads of different trees are even farther apart, Section 5).

(C2) **Exclusive interior points.** An interior point of a trace is neither a pad nor an interior point of another trace, and a trace does not visit a grid point twice.

(C3) **No crossing diagonals.** No two traces use diagonal steps inside the same unit square.

(C4) **Gender rule.** The arrival direction of the trace to v is orthogonal iff $g(v) = \mathrm{F}$ and diagonal iff $g(v) = \mathrm{M}$.

Traces of different edges may meet only at a common pad (the parent's pad). Note that (C2) does *not* forbid traces on adjacent parallel grid lines: two traces may be one grid step apart, which is what gives the drawing its board-like density (Figure 4).

2.4 Two basic properties

Proposition 1 (Planarity)**.** *A drawing satisfying (C2) and (C3) has no crossings: two traces intersect at most in a common pad.*

Proof. Every trace is a union of unit steps between grid points. Two unit steps that share no grid point can intersect only in the interior of a unit square: orthogonal steps run along grid lines and meet grid lines only at grid points, while a diagonal step meets the open interior of exactly one unit square and touches its boundary only at its two end points. Hence an intersection of two steps without a common grid point needs two diagonal steps in the same unit square, either the two crossing diagonals or the same diagonal twice; both are excluded by (C3). A common grid point of two different traces is, by (C2), never interior to either of them unless it is a pad, and a pad is shared only by traces incident to that vertex. □

Proposition 2 (Degree bound)**.** *If $c \geq 2$, a vertex of degree greater than 8 cannot be drawn validly.*

Proof. Consider a pad $p(v)$ and an incident trace. Its point next to $p(v)$ is at Chebyshev distance 1; it cannot be the other end pad because pads are at distance $\geq c \geq 2$ (C1), so it is an interior point. By (C2) the interior points of different traces are distinct, so the $\deg(v)$ incident traces use $\deg(v)$ distinct neighbours of $p(v)$, of which there are only eight. □

The algorithm draws such vertices anyway and reports the conflicts (Section 4.7).

3 Input and validation

Adjacency list. One line per vertex, `Vertex:Neighbour,Neighbour,...`; an isolated vertex is written `D:`. A male vertex is marked with a leading $\star$ *wherever* it appears; unmarked vertices are female. Example (two trees and an isolated vertex):

```
*A:B,*C B:*A *C:*A D:*E,F *E:D F:D G:
```

Algorithm 1: LAYOUTTREE(T, r, S) — greedy routing of one tree

```
root T at r by BFS; compute s(v); sort every ch(v) by s descending, then label
order ← BFS order of T if S.order = bfs else DFS preorder
p(r) ← (0,0);   pend(r) ← κ(r);   B ← {(0,0)}
foreach u ∈ order do
    foreach v ∈ ch(u) do
        ROUTE(u, v)                                          ▷ Algorithm 3
return pads, traces, B, list of forced routes
```

Adjacency matrix. Row i is vertex $\ell(i)$ with labels $\mathrm{A},\ldots,\mathrm{Z},\mathrm{A1},\ldots,\mathrm{Z1},\mathrm{A2},\ldots$; a $\star$ at the start of a row makes that vertex male.

Validation. The input is rejected as a whole (and the current graph is kept) if any of the following occurs; every problem is reported with its line or row: inconsistent gender marks of the same vertex, malformed lines (not exactly one colon, empty or illegal names, a $\star$ not at the start), a vertex with two lines, a neighbour listed twice, self-loops, asymmetric adjacency (list or matrix), entries other than $0/1$, a row of wrong length, more vertices than allowed, and finally cycles. The forest check uses union–find with path halving over the edge list: an edge whose ends already have the same representative closes a cycle. The number of successful unions gives the number of trees, $|V| - \#\text{unions}$.

4 Routing a large tree

Large and medium trees are routed; this section describes the routing of one such tree.

4.1 Root and placement order

Root. If the user names a root in the tree it is used; otherwise the *centre* of the tree is used: a breadth-first search from the first vertex of the component finds an end a of a longest path, a second search from a finds the other end b, and the middle vertex of the a–b path is chosen (the classical tree centre, cf. Jordan [6]). (Small trees are not routed, Section 5.) Starting in the centre minimises the depth of the rooted tree, so the board grows evenly in all directions.

Order. The tree is rooted by breadth-first search, sub-tree sizes $s(v)$ are computed bottom-up, and the children of every vertex are sorted by decreasing $s(v)$ (ties by label), so large sub-trees get the best room. The vertices u are then processed in one of two orders, and for each u all its children are routed one after another:

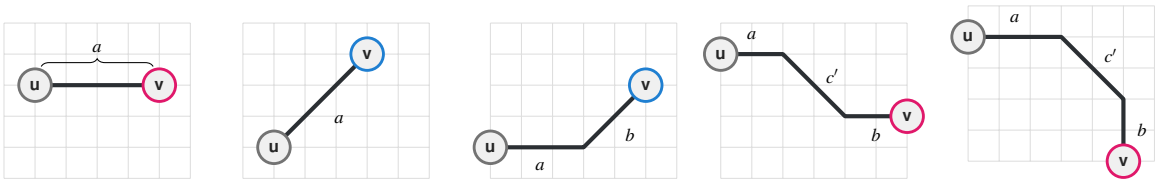
- **BFS order** (default, "generation by generation"): like the metro map, one generation is finished before the next starts, which yields a radial, balanced board.
- **DFS preorder** ("sub-tree by sub-tree"): a sub-tree is finished before its siblings' sub-trees, which clusters families together.

The root's pad is placed at $(0,0)$. Throughout the routing the algorithm maintains the occupied pads, the interior trace points, the used diagonal slots (unit square $\times$ diagonal type), the arrival direction $\delta_{\text{in}}(v)$ of each placed vertex, the bounding box B of everything drawn, and $\text{pend}(w)$, the number of children of a placed vertex w that still need a trace.

4.2 Candidate traces

For the child v of u, the first stage enumerates four families of traces starting at $P = p(u)$ (Figure 5); L is the *maximum segment length* (default 6):

(a) **Straight**: a steps in direction d_1, $1 \le a \le L$. Arrival direction d_1.



(a) Straight, female. (b) Straight, male. (c) One bend, male. (d) Dogleg, female. (e) Chamfered corner.

Figure 5: The candidate families. Segment lengths are independent; the parity of the last segment decides which gender the trace can serve.

(b) **One bend**: a steps in d_1, then b steps in $d_2 = d_1 + s$, $s = \pm 1$, $1 \le a, b \le L$. Arrival direction d_2.
(c) **Dogleg (jog)**: a steps in d_1, c' steps in $d_2 = d_1 + s$ with $1 \le c' \le 3$, and b steps in d_1 again, $1 \le a, b \le L$. Arrival direction d_1. This is the PCB offset: a short sideways shift between two parallel runs.
(d) **Chamfered corner**: a steps in d_1, c' steps in $d_2 = d_1 + s$ with $1 \le c' \le 3$, and b steps in $d_3 = d_1 + 2s$. The trace turns by 90° in two 45° bends, the way real boards turn a corner (no sharp 90° corners). Arrival direction d_3.

All segment lengths are chosen independently, unlike the equal spacing of the metro map. The gender rule (C4) is applied as a filter on the arrival direction: a female child needs an even arrival direction, a male child an odd one. Since $d_1 \pm 1$ has the opposite parity of d_1, a single bend always switches between orthogonal and diagonal, so for every child both the families (a), (c), (d) (last segment with the parity of d_1) and family (b) remain available. Only 45° bends are generated in this stage; a 90° turn without chamfer appears only in the maze router, at a high cost (Section 4.5). Families (c) and (d) can be switched off together ("two-bend traces").

Incremental enumeration. The candidates are generated by a depth-first walk that extends the current path one unit step at a time and undoes the step afterwards. A step that is illegal (Section 4.3) ends the current segment, so all longer candidates sharing that prefix are pruned immediately. Each complete candidate is passed to the evaluation (Section 4.4); a branch-and-bound test (Section 8) stops a walk as soon as no extension can beat the best candidate found so far.

4.3 Legality and the reservation rule

A unit step to a point q in direction k is *free* if q is not a pad, not an interior trace point, not already on the current candidate, and—if k is odd—neither diagonal slot of the unit square crossed by the step is in use. A candidate ending in e is accepted for evaluation if, in addition, e is not an interior trace point and no pad lies within Chebyshev distance $c-1$ of e (this includes $p(u)$, so the child is at least c away from its parent). These rules enforce (C1)–(C4) for every committed trace.

Free directions and landing points. For a placed pad w let

$$\varphi(w) = \#\{k : p(w) + D_k \text{ is free and, for odd } k, \text{ the crossing diagonal slot is unused}\},$$
$$\Lambda(w) = \#\{q : \|q - p(w)\|_\infty = 2,\ q \text{ free},\ \text{no pad other than } w \text{ within distance } c-1 \text{ of } q\}.$$

$\varphi(w)$ counts the exits still available at w (Proposition 2 shows each child needs one); $\Lambda(w)$ counts nearby points where a child pad of w could still land.

Reservation. Greedy routing has a typical failure: a trace for one vertex wanders past another pad w and takes the last exits that w needs for its own, not yet routed children. To prevent this, define $\text{need}(w) = \text{pend}(w)$ for $w \neq u$ and $\text{need}(u) = \text{pend}(u) - 1$ (the trace being routed is one of u's pending children). While a candidate is extended, every new point q is charged to the waiting pads around it (Figure 6):

- a pad w with $\|q - p(w)\|_\infty = 1$ and $\text{need}(w) > 0$ receives one *hit*; the step is refused if $\varphi(w) - \text{hits}(w) < \text{need}(w)$ afterwards;

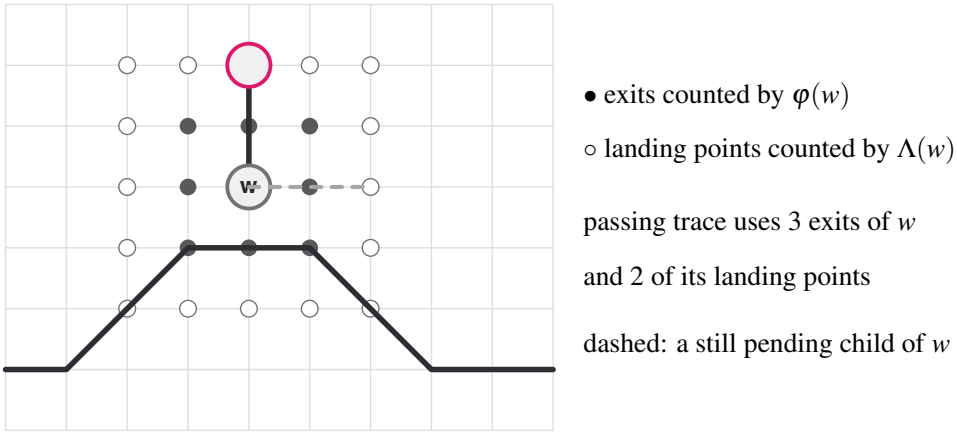


Figure 6: Reservation around a pad w with pending children. A candidate trace passing near w is accepted only if the remaining exits $\varphi(w)$ and landing points $\Lambda(w)$ stay at least need(w).

- a pad $w \neq u$ with $\|q - p(w)\|_\infty = 2$ and need$(w) > 0$ receives one *landing hit*; the step is refused if $\Lambda(w) - \text{landHits}(w) < \text{need}(w)$ afterwards.

φ and Λ are evaluated once per child on the committed state (cached), and hits are counted per candidate and undone with the step. The rule is conservative (a point may be counted for several pads), but it is cheap and removes most dead ends of the greedy order. The ring test is skipped for u itself because the new child pad usually lands at distance 2 from its parent.

4.4 Cost function

Every legal candidate π for the child v of u, ending in e, with $b(\pi)$ bends, receives the cost

$$C(\pi) \;=\; \ell(\pi) \;+\; 0.7\,b(\pi) \;+\; \lambda\,\Delta H(\pi) \;-\; \sigma\,\text{bus}(\pi) \;+\; 60\,\big[\kappa(v) - \min\big(\varphi'(e), \Lambda'(e)\big)\big]^+, \qquad (1)$$

where $[x]^+ = \max(0, x)$ and the cheapest candidate wins (ties go to the first one enumerated). The five terms are:

Length $\ell(\pi)$: number of orthogonal steps plus $\sqrt{2}$ times the number of diagonal steps.

Bends 0.7 per 45° bend; a two-bend trace costs 1.4, so it is chosen only when it pays off.

Compactness ΔH is the growth of the half perimeter $(x_{\max} - x_{\min}) + (y_{\max} - y_{\min})$ of the bounding box B if the candidate is added; the corner points of the trace suffice to compute it. The weight λ is the user's *compactness* (default 1.5).

Bus bonus bus(π) is the number of steps of π that run parallel to an already committed trace on a neighbouring grid line (Figure 7); the weight σ is the user's *bus bundling* (default 0.5). For every grid point the algorithm keeps, per orientation $o = k \bmod 4$ ("–", "\", "|", "/"), the number of committed steps of that orientation touching the point. A step $q \to q'$ of orientation o counts if, for one of the two neighbouring parallel lines (offset $(0, \pm 1)$ for $o = 0$, offset $(\pm 1, 0)$ otherwise), both q and q' shifted onto that line touch a committed step of orientation o. Traces are thereby pulled into dense parallel bundles that turn together, like the buses of a circuit board.

Exit look-ahead The new pad at e will need one exit for each of its $\kappa(v)$ children, and an exit is only useful if the child can also land somewhere. $\varphi'(e)$ is the number of free directions at e and $\Lambda'(e)$ the number of free landing points at Chebyshev distance 2 from e (the quantities φ and Λ of Section 4.3, taken at the end point; points of the candidate itself count as blocked for φ'). The smaller of the two is the number of usable exits, and every child of v beyond it costs 60. The weight is large on purpose: a candidate that starves the children of v is taken only when no other one is legal.

The first four terms describe the trace itself; the fifth is the only term that looks ahead. All weights except λ and σ are fixed.

Table 1: The old nine-term cost and the five-term cost (1) on the same inputs (default settings, one thread). *Test forests*: the ten one-board forests of Section 9. *Crowded*: seven forests of 1000 vertices with large trees of 30–200 vertices and up to 5–7 children per vertex, harder than the target class. Grid points per vertex = board area divided by the number of vertices; free = share of the board's grid points that hold neither a pad nor a trace.

input	cost	grid points per vertex	free	forced routes	verifier problems	time [s]
test forests (10 boards)	old	4.50	56.3%	0	0	4.3
	new	4.53	56.5%	0	0	4.2
25 000 vertices (63 boards)	old	4.66	52.5%	0	1	19.8
	new	4.64	52.4%	0	1	16.0
crowded (7 boards)	old	4.58	53.3%	73	92	87.3
	new	4.53	53.1%	56	60	70.3

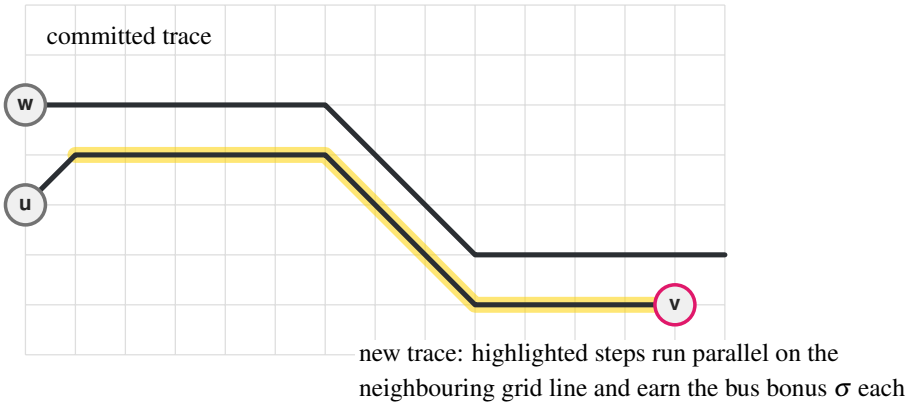


Figure 7: Bus bonus. The trace from u to v follows the committed trace one grid line below it, including the diagonal run, so the two turn together; every highlighted step lowers its cost by σ.

Forced mode. In the forced mode of Section 4.7 a candidate may accept conflicts; there, candidates are compared by their number of conflicts $\chi(\pi)$ first and by $C(\pi)$ second, so no weight is needed to trade a conflict against length.

A simpler cost. An earlier version of the cost had nine terms: besides the five above, a *room look-ahead* that counted free grid points in a window around e and compared them with the size of the sub-tree of v (three further parameters), a bonus of 1 for leaving u in the direction in which u was reached (*line continuation*, as in the metro map), a reward per step away from the root (*radial growth*, used only by one retry variant), and a penalty of 400 per conflict. Removing the room look-ahead alone made crowded trees worse, so its purpose was taken over by Λ' in the exit term, a quantity the reservation rule already computes. Line continuation and radial growth were dropped without a replacement, and the penalty per conflict is replaced by the comparison of the forced mode. Table 1 compares the two versions on the same inputs: the five-term cost leaves 23% fewer forced routes in crowded trees and is up to 19% faster, while board size and free space stay within 1% of the old cost (on the ten test forests the boards are 0.5% larger on average; individual boards move by up to 8% either way).

4.5 Escalation: longer segments and the maze router

The procedure FINDROUTE (Algorithm 2) escalates when no candidate is legal: first the enumeration with L and two-bend traces (if enabled), then with $2L$ without them, then a maze router.

Maze router. The maze router is a Lee/Dijkstra search [7, 4] on states (q,k) = (grid point, direction of arrival at q), so that the cost of a turn is known. From (q,k) the next direction may be k (cost 0), $k\pm1$ (45°, cost 0.7) or $k\pm2$ (90°, cost 3.0, deliberately expensive because a board chamfers its corners); sharper turns

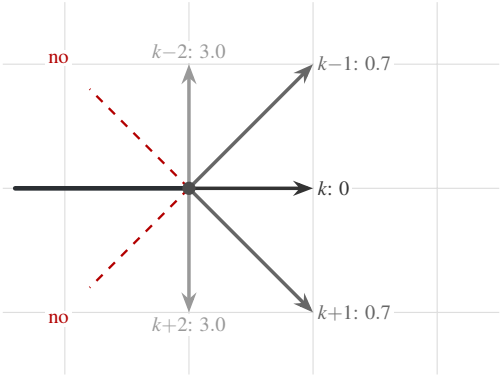


Figure 8: Moves of the maze router from a state reached in direction k (here $k = 0$) with their turn penalties (added to the step length). Turns of 135° and 180° are not allowed; unchamfered 90° turns are expensive.

Algorithm 2: FINDROUTE$(u, v, allowConflicts)$

foreach $L' \in (L, 2L)$ **do**
  enumerate straight, one-bend and (only if $L' = L$ and enabled) dogleg and chamfered-corner candidates with segments $\leq L'$, keeping the cheapest legal one by (1) (with branch and bound)
  if *found* **then return** *it*
$\pi \leftarrow$ MAZEROUTE(strict) **or** MAZEROUTE(non-strict); **if** π **then return** π
if *not* *allowConflicts* **then return** *none*
enumerate straight and one-bend candidates with segments $\leq \max(2L, w_B + h_B + 4)$ in *forced mode* (conflicts with traces allowed, pads never crossed, no reservation; fewest conflicts first, then (1))
if *found* **then return** *it*
return *straight trace from* $p(u)$ *to* $(x_{\max}(B) + c + 1,\ y(p(u)))$ ▷ last resort

are not allowed (Figure 8). A move costs its step length plus the turn cost plus a soft reservation cost of 2 for every waiting pad at distance 1 and 1 for every waiting pad $w \neq u$ at distance 2, minus the bus bonus of the step (capped so that every move still costs at least 0.2, which keeps Dijkstra's algorithm correct); the hard reservation test of Section 4.3 applies with a single hit. The goal test accepts the first popped state whose arrival direction has the right parity and whose point respects the clearance; in the *strict* run it must also offer at least $\kappa(v)$ free directions. A strict run is tried first, then a non-strict one, each limited to 60 000 expansions.

4.6 Rip-up and reroute

If FINDROUTE fails, the space around u is usually blocked by traces that were routed earlier but could have gone elsewhere. As in PCB routers [8], such traces are removed and rerouted (Algorithm 3). A committed trace to a vertex x is *movable* if $x \neq u$ and none of x's children is placed yet (so moving x drags nothing along) and the trace comes within Chebyshev distance 4 of $p(u)$. Movable traces are sorted by that distance. For $m = 1, \ldots, \min(6, \#\text{movable})$ the state is saved, the m nearest traces are ripped up (pads removed, pend of their parents increased), v is routed, and then the ripped vertices are routed again in their original order. If every one of them succeeds the result is kept; otherwise the saved state is restored and m is increased.

4.7 Forced routes

Only when everything above fails is a conflicting trace accepted: the forced mode of Algorithm 2 allows a trace to overlap other traces or cross their diagonals (it never passes through a pad and the reservation rule is off); the candidate with the fewest conflicts wins, and among those the cheapest by (1). If even that finds nothing, a straight horizontal trace to a new pad just right of the bounding box is drawn. Every forced route is listed in the status line and its conflicts are found and marked by the verification. Forced routes are the

Algorithm 3: ROUTE(u, v) with rip-up and reroute

```
if π ← FINDROUTE(u, v, false) then commit π; return
M ← movable traces within distance 4 of p(u), nearest first
for m ← 1 to min(6, |M|) do
    save state; rip up the first m traces of M
    if π ← FINDROUTE(u, v, false) then
        commit π
        if every ripped vertex x (in order) gets a route π_x by FINDROUTE(par(x), x, false), committed
          at once then return
    restore state
commit FINDROUTE(u, v, true) and record u → v as forced
```

expected outcome for vertices of degree above eight (Proposition 2) and can occur in very crowded regions of large trees (Section 9).

4.8 Multi-start

Greedy routing depends on the root and the two weights. LAYOUTTREEBEST therefore runs LAYOUTTREE (without the squeeze pass) with a list of (root, settings) attempts and keeps the result with the fewest forced routes, breaking ties by the board it needs at the chosen aspect ratio α: with bounding box $w \times h$, the area of the smallest α-box around it, $\max(w, \alpha h)^2/\alpha$ (plain area would favour long thin strips). While routes conflict, attempts continue for up to 4 seconds. Once an attempt is conflict-free, further attempts for a smaller board run only within the budget T_{choose}, and only if they are expected to finish within it (judged by the duration of the previous attempt). By default $T_{\text{choose}} = 0$: in the target forests the board size is set by the packing of all trees, and extra attempts showed no gain. The roots are the chosen root and, if the user did not fix it and the tree has more than two vertices, the vertex of highest degree, the first vertex of the component, and the neighbours of the centre. The attempts are, in this order:

1. for every root: the user settings S, then the *loose* variant ($\lambda/2$);
2. for every variant of ($\sigma = 0$), ($\lambda = 0.25$), ($L+3$ capped at 12 with $\lambda/2$), (the other placement order), for every root.

The squeeze pass of the next section then runs on the chosen attempt itself; no attempt is laid out twice.

4.9 Squeeze (rip-up compaction)

Greedy routing leaves slack at the border of a tree: a single vertex or trace can hold a whole row or column of the bounding box. The squeeze pass removes such slack in the manner of layout compaction in VLSI and PCB design. It repeatedly takes one side of the bounding box B (right, left, bottom, top in turn), collects every vertex whose pad or incoming trace touches that side, and rips up the sub-trees rooted at these vertices. It then routes them again, in the original order and with the normal router including rip-up, inside B shrunk by one grid step on that side (Figure 9). A shrink is kept only if no route had to be forced; otherwise the saved state is restored. The pass ends after four consecutive failures, or when the next step would exceed the time budget T_{squeeze} (default 800 ms for a single board and 150 ms per board when the forest is divided into boards, shared by all routed trees of the board in proportion to their size; a step is started only if the previous one suggests it fits). A side is skipped if the root touches it, if the box is already narrower than 2 steps in that direction, or if more than $\max(12, 0.3\,n)$ vertices would have to be ripped up.

Proposition 3. *The squeeze never enlarges the bounding box, never adds a forced route, and keeps every drawing valid.*

Proof. During re-routing every point outside the shrunk box is treated as blocked, so all new pads and traces lie inside it. Everything that is not ripped up does not touch the removed side, by the choice of the ripped

Algorithm 4: SQUEEZE(T_{squeeze})

```
fails ← 0;   side ← right
while fails < 4 and the next step is expected to fit into T_squeeze do
    X ← vertices whose pad or incoming trace touches side of B
    if root ∈ X or |sub-trees of X| > max(12, 0.3n) then
        fails ← fails + 1
    else
        save state; rip up the sub-trees of X (deepest first); limit ← B shrunk by 1 on side
        route the ripped vertices again in the original order, inside limit
        if a route was forced then restore state; fails ← fails + 1
        else fails ← 0
    side ← next side (right, left, bottom, top)
```

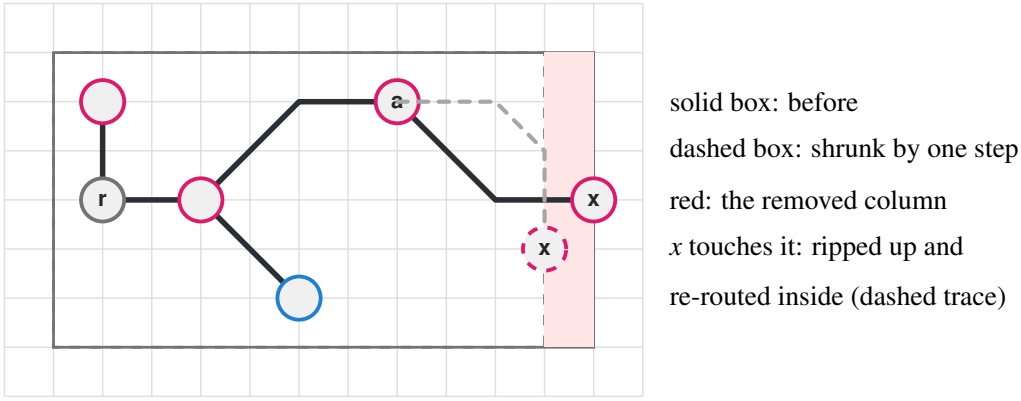


Figure 9: One squeeze step on the right side. The sub-tree of x is the only one touching the right column; it is routed again inside the box shrunk by one column, here with a chamfered corner.

vertices, so the box shrinks by one. All new traces are produced by the normal router and obey the same legality rules (Section 4.3); if any of them had to be forced, the complete previous state is restored. □

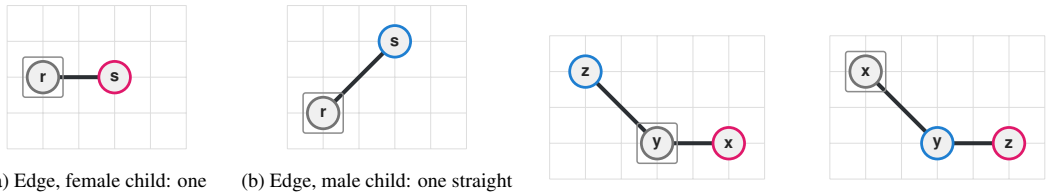
5 Forest packing

5.1 Tree classes

The trees of the forest are split by their number of vertices n_T into *large* ($n_T \geq 9$), *medium* ($4 \leq n_T \leq 8$) and *small* ($n_T \leq 3$) trees. Large and medium trees are routed as in Section 4; the time budgets of the squeeze and of the extra attempts are shared by these trees only, in proportion to their size. Small trees are not routed at all: their drawings come from templates (Section 5.2) and cost $O(1)$ time each.

5.2 Straight templates for small trees

A small tree is an isolated vertex, a single edge, or a path x–y–z. Its *variants* are all drawings in which every edge is one straight trace: from the parent the trace runs in a direction whose parity matches the gender of the child (orthogonal into a female, diagonal into a male), with length c, or, for a single edge only, also $c+1$. Every vertex may be the root (or only the user's root node, if it lies in this tree). Each variant has a *rank*, its total extra length (0 for the shortest traces). Variants with the same geometry (labels ignored, after translation) are merged, keeping the lowest rank; a single edge has 4 variants, a path of three vertices at most 48 (Figure 10).



(a) Edge, female child: one straight trace.

(b) Edge, male child: one straight diagonal.

(c) Path rooted at its middle.

(d) Path rooted at an end.

Figure 10: Straight-only templates of small trees (framed: root). Each edge is one straight trace of length $c = 2$ whose direction follows the gender of the child; no template contains a bend. A single edge also has variants of length $c+1$ (rank 1), used only if no shortest one fits.

Proposition 4. *Every small tree has a valid drawing with only straight traces of length c that obeys the gender rule, for any choice of root. In particular a tree of two vertices is always drawn as one straight trace without bends.*

Proof. For a single edge $r \to s$ choose any direction D_k whose parity matches $g(s)$ and put $p(s) = p(r) + cD_k$; the two pads are at distance c and the trace has no other trace to meet. For a path rooted at its middle y, choose directions $D_a \neq D_b$ with the parities required by the two ends; the rays meet only in y, and since D_a and D_b differ by at least 1 in some coordinate, the end pads are at Chebyshev distance at least c. For a path rooted at an end, $x \to y$ in D_a and $y \to z$ in $D_b \neq -D_a$ (four choices of D_b remain); then $p(z) - p(x) = c(D_a + D_b)$ has a coordinate of absolute value at least c because $D_a + D_b \neq 0$, the two rays share only y, and their diagonal steps at y lie in different unit squares (that would need $D_b = -D_a$). □

5.3 Rotations and reflections

Lemma 1. *Let g be one of the eight symmetries of the grid (rotations by multiples of $90°$ and reflections in the axes and diagonals). Applying g to all pads and traces of a valid drawing gives a valid drawing that obeys the gender rule.*

Proof. g is an integer orthogonal map: it maps grid points bijectively to grid points, preserves Chebyshev distances (so (C1) holds), maps unit octilinear steps to unit octilinear steps, orthogonal ones to orthogonal ones and diagonal ones to diagonal ones (so (C4) holds), keeps distinct points distinct ((C2)), and maps the two diagonals of a unit square to the two diagonals of the image square ((C3)). □

Every routed tree therefore has up to eight *variants* for free (duplicates of symmetric drawings are removed).

5.4 The board grid

Packing works on one board stored as flat typed arrays over a rectangle of grid points that grows on demand (by at least 32 steps per side, copying row by row): *occ* marks occupied points (pads and trace points), *pz* the *pad zone* (points closer than c to a placed pad), *dg* the used diagonal slots of every unit square, and $near(q)$ the number of occupied points within Chebyshev distance 2 of q. A variant is stored as its *footprint*: the lists of its pads, of all its points, and of its diagonal slots. A placement at offset o is *legal* if no pad of the footprint lies in the pad zone, no point is occupied and no diagonal crosses a used one; each test is one array lookup, so a legality check costs $O(|\text{footprint}|)$. Cells outside the allocated rectangle are empty.

Proposition 5. *Every packed board satisfies (C1)–(C4).*

Proof. Inside a tree the constraints hold by construction (Sections 4 and 5.2) and are preserved by translation and, by Lemma 1, by the symmetries. Between trees, a legal placement has no pad closer than c to another tree's pad, no point in common with another tree, and no crossing diagonals, which are exactly (C1)–(C3). □

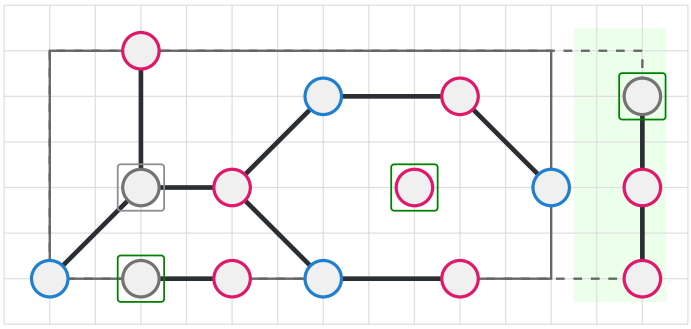


Figure 11: Nestling. Small trees (green frames: their roots) are placed into free cells inside the board first; only when none fits does the board grow, by the strip that enlarges it least.

5.5 Order and score

The largest tree (by footprint size) is placed first, in its original orientation. Then come the other large trees, then the medium trees, then the small trees by decreasing size (3, 2, 1). For a placement that turns the board box B into B' of $W \times H$ steps, the score is lexicographic, smaller first:

$$\big(f(B') = \max(W,\ \alpha H),\ \ (W+1)(H+1),\ \ \text{rank},\ \ -\text{contact}\big),$$

where α is the board aspect (width/height, default 1.6) and $\text{contact} = \sum_{q \in \text{footprint}} \min(near(q), 4)$ measures how snugly the variant sits against what is already drawn. Rank and contact are used only for medium and small trees (*nestling*); for large trees, ties keep the scan order.

Large trees. All variants and all offsets that keep the tree within distance c of the board are scanned; a candidate is tested for legality only if its (f, area) could beat the best one found so far.

5.6 Nestling medium and small trees

A variant can only be placed where its first pad lands on a free cell (not occupied and outside every pad zone). The search is therefore driven by free cells instead of all offsets, in two passes (Figure 11, Algorithm 5):

1. **Inside.** The free cells inside B are listed once; every variant is anchored on each of them and kept if it lies completely inside B and is legal. The board does not grow, so the best candidate has the lowest rank and, among those, the most contact. If there is one, it is taken.
2. **Grow.** Otherwise the board must grow. The boxes B pushed out by $k = 1, 2, \ldots, e + c + 1$ steps on one side (e = the largest variant extent) are sorted by (f, area). In each box only anchors in the new strip and within distance e of it are tried, and only placements that reach into the strip (all others failed in pass 1). The first box with a legal placement decides; in it, rank and then contact pick the winner.

Pass 2 makes the next small trees cheap again: they find room in the new strip during pass 1. Because the shortest variants have rank 0, a single edge uses its longer variant only when no shortest one fits anywhere in the board; on the ten test forests of 100–1000 vertices (Section 9), 494 of the 495 single edges have length c and one has length $c + 1$; none has a bend.

Finally the board is translated so that its box starts at $(1, 1)$, leaving a margin of one grid step.

6 Verification

After packing, the drawing is checked by a procedure that shares no data with the construction. It re-derives everything from the final coordinates:

1. **Pads**: two pads on the same point; every pair of pads closer than c (all pairs, $O(n^2)$).
2. **Traces**: start and end on the pads of the edge; every step is a unit octilinear step; no interior point is a pad; no interior point is used by two traces (overlap); no unit square contains both diagonals (crossing).

Algorithm 5: NESTLE(variants V) — placement of a medium or small tree

```
F_in ← free cells inside B;   best ← none
foreach v ∈ V, q ∈ F_in do
    o ← q − first pad of v
    if v + o ⊆ B and legal(v, o) then
        keep (v, o) if better by (rank, −contact)
if best then return best
foreach box B′ = B pushed out by k on one side, by increasing (f(B′), area) do
    if best and B′ is worse than best then break
    foreach v ∈ V, free anchor q in the strip of B′ (or within distance e of it) do
        o ← q − first pad of v
        if v + o ⊆ B′, v + o ⊈ B and legal(v, o) then
            keep (v, o) if better by (rank, −contact)
return best
```

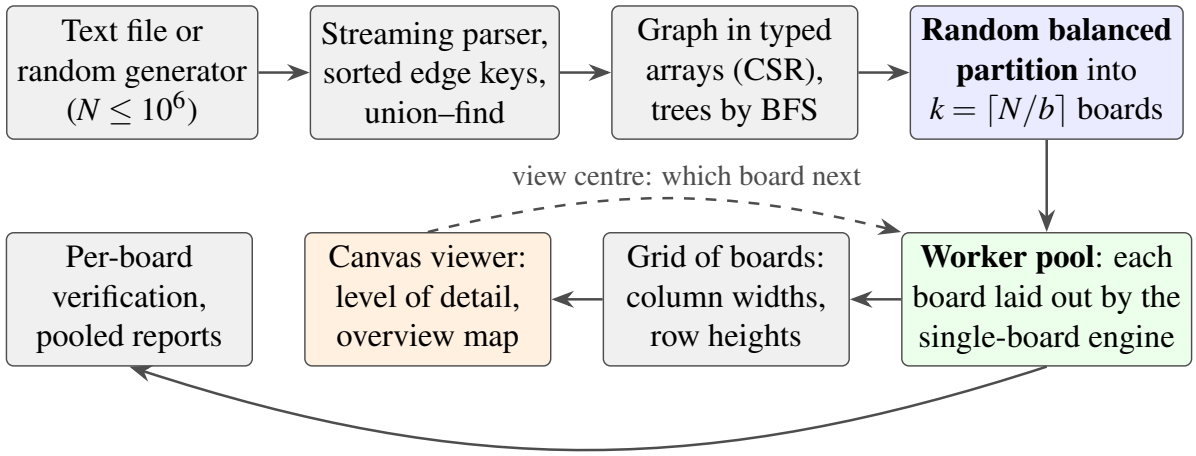


Figure 12: Pipeline for many boards. The partition and the parallel layout of the boards (blue, green) are the core; the viewer (orange) decides which board is laid out next.

3. **Gender**: the last step of the trace to v is diagonal iff $g(v) = \mathrm{M}$.

Problems are listed in three reports and circled in the drawing. By the construction rules, a layout without forced routes passes all checks, and by Proposition 1 it is crossing-free. Because all coordinates are integers, the checks are exact.

7 Forests of up to a million vertices

Routing one tree takes time that grows faster than linearly with its size (Figure 17), and one board for a million vertices could neither be routed in reasonable time nor be displayed. For large forests the algorithm therefore never routes more than one *sub-forest* on a board: the forest is divided into sub-forests of a chosen number of vertices, every sub-forest is drawn by the one-board algorithm of Sections 4–6, and the boards are placed in a grid. Figure 12 shows the pipeline; every step works on flat integer arrays, so memory and time grow linearly with the input.

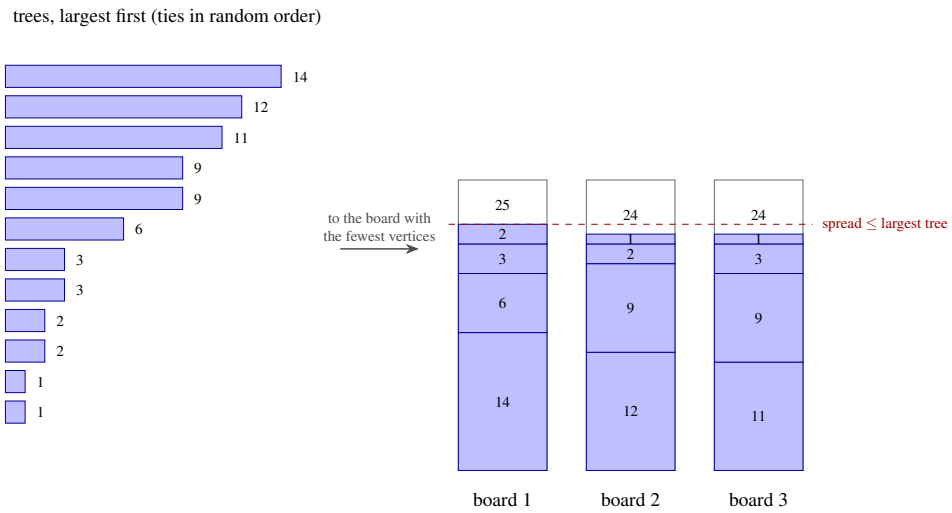


Figure 13: Partition by number of vertices with $k = 3$. Trees are taken largest first; each goes to the board with the fewest vertices so far. The final loads (25, 24, 24) differ by less than the largest tree.

7.1 Reading a million lines

The input has the same adjacency-list format and the same checks as for one board, but it is read in a single pass. Every vertex name is mapped to an integer identifier with one hash table, and every directed neighbour entry (u, v) is appended to two growable integer arrays. The checks that need pairs of entries are done after the pass: every entry becomes the number $uN + v$, and these numbers are sorted. A neighbour listed twice then appears as two equal neighbours in the sorted order, and a missing reverse entry is found by binary search for $vN + u$. Each edge is kept once, from its smaller end. Finally union–find with path halving rejects cycles. With $D \le 2|E|$ directed entries, reading a text of length L takes $O(L + D \log D)$ time.

7.2 The forest in arrays

The forest is stored in compressed sparse row form: an offset array of $N + 1$ entries and an array of the $2|E|$ neighbours. The trees are found by breadth-first search in input order and stored as one array of vertices, tree after tree, with an offset array; the largest degree and the largest tree are recorded. This takes $O(N)$ time and about $4(3N + 2|E|)$ bytes.

7.3 Random partition into boards

Let b be the wanted number of vertices per board. It can be *any* number (from 10 to 20 000; 400 by default), so the user decides how large every board is and therefore how many boards there are: $k = \min\bigl(T, \lceil N/b \rceil\bigr)$, where T is the number of trees. A tree is never cut. The trees are first shuffled (Fisher–Yates with a seeded random generator, so a partition can be reproduced) and then distributed in one of two ways:

By number of vertices (default). The shuffled trees are stably sorted by decreasing size, so trees of equal size stay in random order, and each tree goes to the board that has the fewest vertices so far (a min-heap keyed by load and board index). This is greedy list scheduling in the largest-first order [5] (Figure 13).

By number of trees. The shuffled trees are dealt round-robin, so the numbers of trees per board differ by at most one.

Proposition 6. *In the partition by number of vertices, the loads of any two boards differ by at most $s_{\max}$, the number of vertices of the largest tree.*

Proof. Let board i have the largest final load and let t be the last tree assigned to it, of size $s_t \le s_{\max}$. When t was assigned, board i had the smallest load ℓ of all boards, so every board j had load at least ℓ at that moment, and loads never decrease. Hence the final load of every board is at least ℓ, while the final load of board i is $\ell + s_t \le \ell + s_{\max}$. □

Why divide by vertices. Dividing by vertices gives boards of *nearly the same size*: in the target forests the largest tree has at most a few dozen vertices, so by Proposition 6 every board receives almost exactly b vertices; in the measurements every one of 2500 boards received exactly 400 (Section 9). Dividing by trees is quite unlike this. Because large and small trees are mixed at random, the round-robin deal gives every board the same *number* of trees but very different numbers of *vertices*: a board that happens to receive many large trees is several times bigger than one that receives mostly small trees (from 130 to 738 vertices for $b = 400$ in our measurements). Since the routing time of a board and the area of its drawing grow with its number of vertices, near-equal boards also mean near-equal work for the parallel threads and near-equal cells in the grid of boards (Section 7.7). Sorting costs $O(T \log T)$, the heap $O(T \log k)$, and collecting the vertices of every board $O(N)$.

7.4 One board per sub-forest

For every board a *job* is built: the names and genders of its vertices and its edges in local numbering. The job is laid out by exactly the one-board algorithm of Sections 4 to 6: small-tree templates, routing of the large and medium trees with squeeze, packing with nestling, and the independent verification. Only two settings differ from a single board: the squeeze budget is 150 ms per board, and the user's root node is used only on the board that contains it. The result is returned as flat arrays: pad coordinates, the end vertices of every trace, the points of all traces in one array with an offset array, the number of bends, the roots, and the positions of reported conflicts.

7.5 Parallel layout ordered by the view

The boards are independent, so they are laid out in parallel by W worker threads (by default one fewer than the number of processor cores, between 1 and 8). Every worker holds its own copy of the one-board algorithm and lays out one board at a time; jobs and results are passed between threads without copying. A run counter discards results of an earlier run after the settings or the forest change.

The order of the work follows the user: whenever a worker becomes free, it receives the queued board that is closest to the centre of the current view, measured in rows and columns of the grid of boards and found by a search in rings around the centre (Figure 14). The part of the forest that is being looked at is therefore drawn first, and moving the view redirects the remaining work.

7.6 The grid of boards

The boards are placed row by row. The number of columns is chosen so that the grid has the aspect ratio of the window: $c = \max\bigl(1, \min(k, \mathrm{round}(\sqrt{k\, a_{\mathrm{win}} / a_{\mathrm{board}}}))\bigr)$, unless the user fixes the number of boards per row. Every column is as wide as its widest board and every row as high as its highest board, with a gap of $\max(4\,\mathrm{pitch}, 64)$ pixels; each board sits centred in its cell inside a numbered frame. A board that is not laid out yet uses an estimated size: area $V \cdot \hat{p}$ grid points with aspect $\hat{a}$, where $\hat{p}$ (grid points per vertex, initially 9 for clearance 2) and $\hat{a}$ are vertex-weighted averages over the boards finished so far. When a board finishes, the column widths and row heights are recomputed and the view is re-anchored at the same relative position inside the same cell, so the picture does not jump.

7.7 Using the space of the grid

The one-board algorithm already packs each board densely (Section 5). The grid adds a second level of packing, and it uses its space well *indirectly*, through the partition rather than through any search:

- every board holds almost exactly b vertices (Proposition 6), and the one-board algorithm needs about the same number of grid points per vertex on every board, so all boards have almost the same area;
- every board is laid out towards the same aspect ratio α, so all boards have almost the same shape;
- a column is as wide as its widest board and a row as high as its highest board, so equal boards fill their cells almost completely.

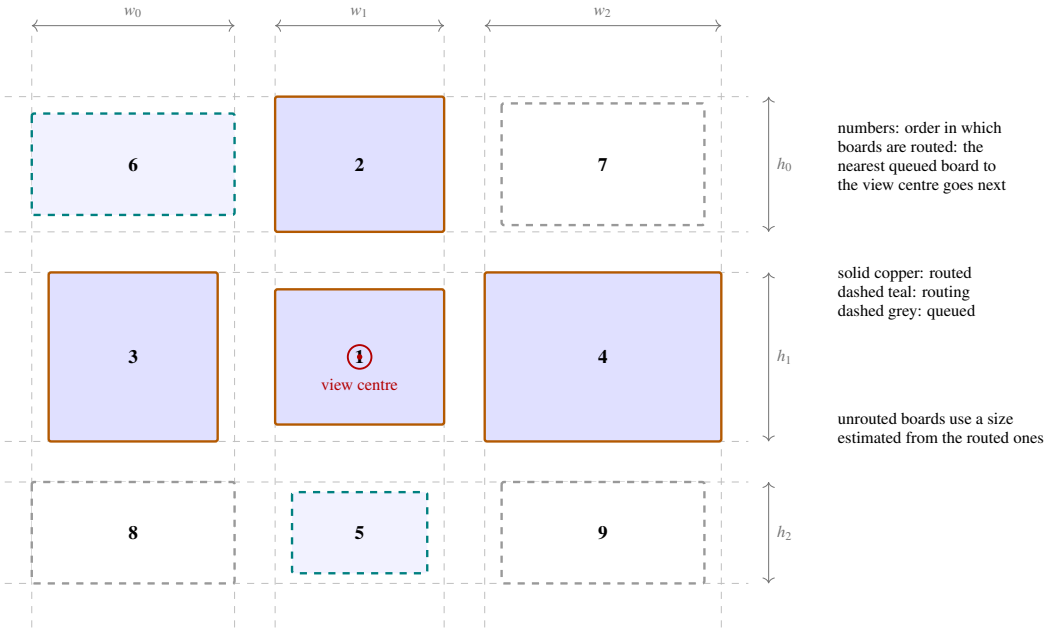


Figure 14: Grid of boards. Column widths w_c and row heights h_r are set by the largest board in each; the numbers show the order in which boards are laid out: the queued board nearest to the view centre goes next.

The grid of boards therefore behaves like a tiling with nearly identical tiles, and only the fixed gaps between the boards remain empty. For forests of 400 or more vertices this uses the space as well as one ever larger board would, at a fraction of the cost: on one board the nestling of the small trees and the verification take time quadratic in the number of vertices (Section 8), and the fixed time budgets of the squeeze and the retries are spread over ever more trees, while the grid keeps every board at the size where the one-board algorithm is dense and fast. Dividing by trees would break this: boards of very different sizes would leave large empty parts of the cells that the biggest board of each row and column determines. Section 9 measures how much of each cell the boards fill.

7.8 Level of detail

A frame draws only the boards whose cells intersect the view; inside a board, pads and grid points outside the visible range are skipped. With p_s the number of screen pixels per grid step, a finished board is drawn as (Figure 19):

- $p_s < 1.6$: a cached *thumbnail* (at most 320 pixels on its longer side), with all traces drawn as one path and the pads as dots coloured by gender; at most 30 ms per frame are spent making new thumbnails, the rest follow in the next frames;
- $1.6 \le p_s < 4$: traces from a cached path of their corner points (kept for the 500 most recently used boards) and pads as filled dots;
- $p_s \ge 4$: traces with outline and pads with gender rings; from $p_s \ge 6$ also the roots and the optional grid, from $p_s \ge 8$ the highlights, and from $p_s \ge 12$ the pad names (if the font would be at least 4 pixels).

Boards that are still queued or being laid out are drawn as dashed frames. An overview map shows the whole grid and the current view; the reports of all boards are pooled, each message prefixed with its board number.

7.9 Random forests

For testing, random forests of the target class can be generated. Large trees have a uniform size in $[\ell_{\min}, \ell_{\max}]$ (mean A), small trees 1–3 vertices (mean 2). Every new vertex of a tree is attached to a random earlier vertex that has fewer than the allowed number of children (at most 24 tries, otherwise to the newest vertex). To give

a share f of the vertices to small trees, the next tree is small with probability

$$q = \frac{fA}{2(1-f)+fA}, \qquad \text{because} \qquad \frac{2q}{2q+A(1-q)} = f;$$

$f = \frac{1}{2}$ gives the half-large, half-small forests of the target class.

8 Running time

Enumeration. Let n be the number of vertices of a routed tree and L the maximum segment length. The enumeration for one child visits at most $8L$ straight, $16L^2$ one-bend and $96L^2$ two-bend candidates (middle segment ≤ 3, dogleg or chamfered corner). Extending a path costs $O(L)$ for the self-intersection test plus $O(1)$ for the reservation and the bus bonus, so the first stage costs $O(L^3)$ per child and $O(nL^3)$ per tree. The second stage ($2L$) multiplies this by a constant. The maze router is bounded by 60 000 expansions of five moves each; rip-up repeats FINDROUTE at most $\sum_{m=1}^{6}(m+1) = 27$ times for a failing child. Multi-start and squeeze are bounded by their time budgets.

The following three speed-ups change *no* drawing. Branch and bound alone makes the routing about four times faster: on ten random trees of 32–140 vertices, routed with fixed roots and settings, the total time fell from 1.91 s to 0.49 s with identical pads and traces.

1. Branch and bound. Let G be the number of bus steps collected so far. For a partial trace with length ℓ, b bends and box growth ΔH, the cost of every completion satisfies

$$C(\pi) \;\geq\; \underline{C} = \ell - \sigma G + 0.7\,b + \lambda\,\Delta H,$$

because $\underline{C}$ is (1) without the exit term, which is non-negative.

Proposition 7. *If $\sigma \leq 1$ (the default is 0.5), $\underline{C}$ never decreases when the partial trace is extended. Hence a walk can be stopped as soon as $\underline{C} \geq C(\pi^*)$ for the best candidate π^* found so far, without changing the result. In forced mode the same holds for the pair $(\chi, \underline{C})$ in lexicographic order.*

Proof. An extra step adds its length (1 or $\sqrt{2}$) to ℓ and at most $\sigma \leq 1$ to σG, so $\ell - \sigma G$ does not decrease; b and ΔH (the box only grows) do not decrease either. Every completion of a stopped walk therefore costs at least $C(\pi^*)$ and cannot replace π^*, which wins ties as the first one found. In forced mode, the number of conflicts χ of a partial trace never decreases either. □

For $\sigma > 1$ the test is switched off. Because no term of (1) can decrease along a walk except through the bus bonus, this bound is exact for every setting the user can choose up to $\sigma = 1$.

2. Reservation table. Instead of scanning the 5×5 window around every step for waiting pads (Section 4.3), the waiting pads are entered once per child into two tables, *near* (distance 1) and *ring* (distance 2, without u), keyed by grid point. A step then needs one lookup. Building the tables costs 24 entries per waiting pad.

3. Lazy look-ahead cache. The free directions and the free landing points around an end point depend only on the committed drawing, so they are computed once per end point and child, and only for candidates whose cost without the exit term already beats the best one. The diagonal slots are computed without allocating objects.

Packing. A small tree has at most 48 variants with at most 5 footprint points. Pass 1 costs $O(|F_{\text{in}}| \cdot |V|)$ legality checks of constant size, where $|F_{\text{in}}|$ is the number of free cells inside the board, which shrinks as the board fills; pass 2 examines one or two strips in practice. With $|B|$ the number of board cells, packing k_s small trees costs $O(k_s\,|B|)$ in the worst case, against $O(k_s\,|B|\,|V|)$ for a scan over all offsets. Placing a large tree costs $O(8\,|B|\,|\text{footprint}|)$ in the worst case. Verification costs $O(n^2 + \sum_e |\pi(e)|)$.

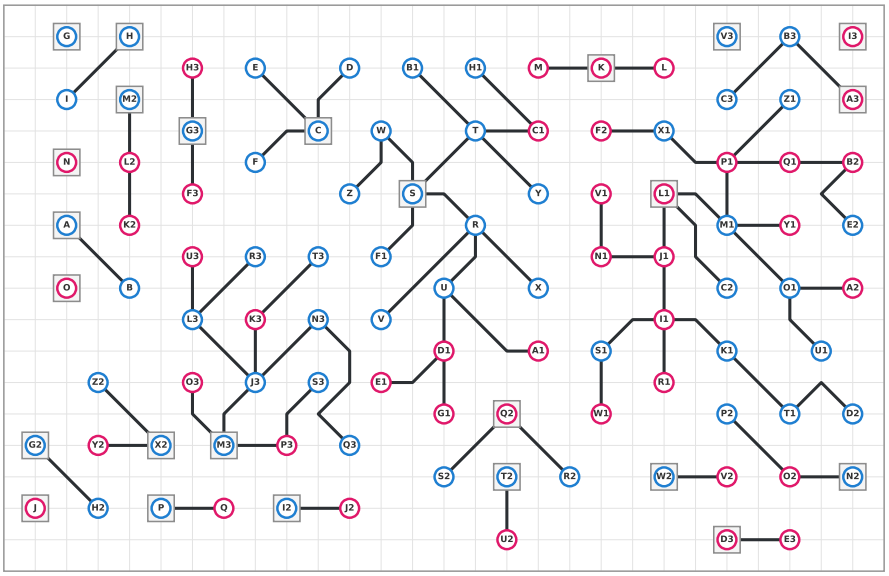


Figure 15: The test forest of 100 vertices on one board (26×16 grid steps, 4.6 grid points per vertex, no conflicts). Magenta rings: female, blue rings: male, framed: roots.

Many boards. With N vertices, $|E| < N$ edges, T trees and k boards, parsing costs $O(L + N \log N)$, the graph $O(N)$, the partition $O(T \log T + N)$, and building the jobs $O(N)$. Each board holds about b vertices (Proposition 6), so its layout costs $O(f(b, s_{\max}))$ for a function f that depends only on b and the largest tree, not on N; with time budgets for the squeeze and the retries, the routing of all boards costs $O(k \cdot f(b, s_{\max})) = O(N)$ for fixed b and $s_{\max}$, divided among W threads. Drawing a frame costs $O(\text{visible boards})$ for thumbnails and $O(\text{visible pads and traces})$ close up. The whole pipeline is therefore linear in N for bounded tree sizes, which is what the measurements show. A forest with one huge tree is the exception: that tree is routed on a board of its own and dominates the time.

9 Results

All measurements use the default settings, on a machine with two processor cores; one board was timed as the median of three runs of the routing engine in Node.js, many boards in the viewer (Chromium) with two worker threads. The test forests belong to the target class. For one board there are ten forests of 100–1000 vertices: in the four forests of 1000 vertices exactly half of the vertices lie in large trees and half in small trees (three of them have no isolated vertices); in the six smaller ones 53–61% lie in large trees, 34–45% in small trees and at most 6% in medium trees. For many boards there are forests of 2500 to 10^6 vertices made by the generator of Section 7, with large trees of 9–60 vertices, at most four children per vertex, and about a quarter of the vertices in small trees.

9.1 One board

Figure 15 shows the test forest of 100 vertices drawn on one board (three large trees, one medium tree, 21 small trees): straight single edges and paths fill the gaps around the large trees, and no 2-vertex tree has a bend. Figure 16 summarises the ten forests of 100–1000 vertices. Every board was drawn without any problem: no collision, no forced route, every arrival following the gender rule, and none of the 495 trees of two vertices bent. A board of 1000 vertices takes 0.6–0.8 s. The boards need 4.3–4.7 grid points per vertex; the bound for clearance 2 is about 4, since a pad keeps the eight cells around it free of other pads. Between 64% and 76% of the traces are straight, 1–9% have two bends, and 5–18% of all trace steps run in parallel buses.

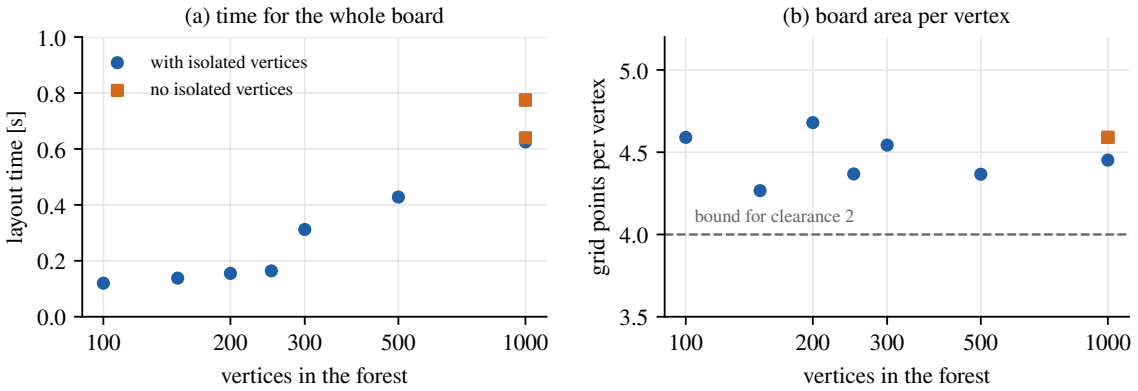


Figure 16: One board on the ten test forests of 100–1000 vertices: (a) time for the whole board, (b) grid points of the board per vertex.

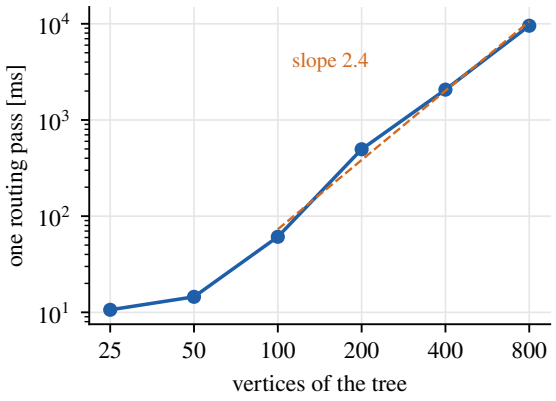


Figure 17: Time of one routing pass of a single tree against its number of vertices (log–log, mean of three trees).

Size of one tree. Figure 17 shows one routing pass for single random trees of 25–800 vertices (without squeeze, three trees per size). Up to 50 vertices a pass takes under 20 ms; beyond 100 vertices the time grows like $n_T^{2.4}$ and forced routes appear: at 100 vertices the three trees had 2, 6 and 1 forced routes after one pass (which the multi-start and the squeeze then try to remove), at 400 vertices about 150 per tree. This is the reason for dividing large inputs into boards of a few hundred vertices.

9.2 Partition

With $b = 400$ vertices per board, the partition by vertices gave boards of 333–334 vertices for 1000 vertices (3 boards), 357–358 for 2500 (7), 396–397 for 25 000 (63), and exactly 400 vertices on every board for 10^5, $2.5 \cdot 10^5$, $5 \cdot 10^5$ and 10^6 vertices (250 to 2500 boards), far inside the bound of Proposition 6 (the largest tree has 60 vertices). The numbers of trees per board varied only from 57 to 62. The partition by trees balances the tree counts instead (58–59 trees per board for 10^6 vertices), but then the board sizes range from 130 to 738 vertices, a factor of more than five; the partition by vertices is therefore the default.

9.3 A million vertices

Table 2 and Figure 18 report the many-board algorithm on the test forests of 10^3 to 10^6 vertices ($b = 400$, partition by vertices, two worker threads on two cores). Figure 1(b) shows 10^5 vertices on 250 boards. Reading the largest input (18.6 MB, 10^6 lines) takes 2.0 s; the rest of the time is spent on routing. From 25 000 vertices on, the time grows linearly with N: about 3500 vertices are drawn per second on two cores, and 10^6 vertices on 2500 boards are finished after 4.8 minutes. The sum of the routing times of all boards is

Table 2: Many boards on the test forests ($b = 400$, partition by vertices, two worker threads on two cores). Conflicts are those that remain after local repair and are reported by the verification of each board.

vertices	boards	grid of boards	vertices per board	grid points per vertex	time [s]	conflicts
1 000	3	2×2	333–334	4.1	0.6	0
2 500	7	4×2	357–358	4.6	1.3	0
25 000	63	9×7	396–397	4.4	7.7	1
100 000	250	17×15	400	4.4	28.8	9
250 000	625	28×23	400	4.4	73.9	17
500 000	1 250	38×33	400	4.4	144.8	34
1 000 000	2 500	55×46	400	4.4	287.1	54

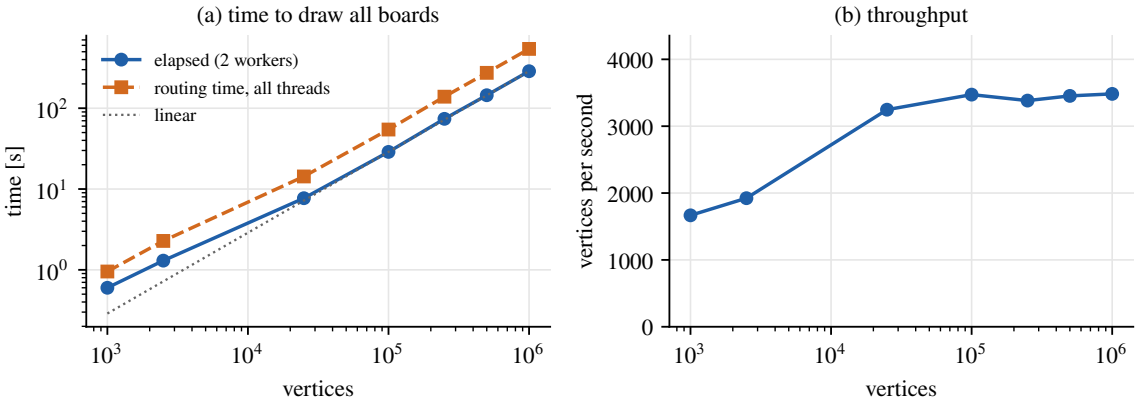
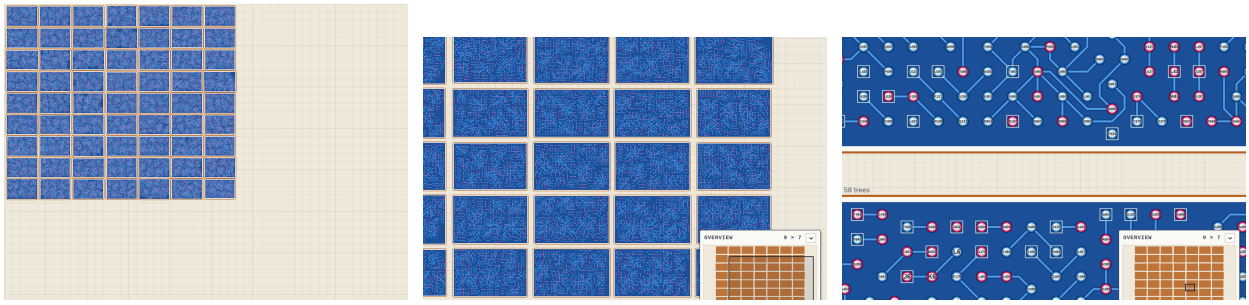


Figure 18: Many boards: (a) elapsed time and the summed routing time of all boards, with a linear reference line; (b) vertices drawn per second.

about twice the elapsed time, so both threads are kept busy; with one worker thread instead of two, the 10^5 vertices take 47.0 s instead of 28.8 s (a speed-up of 1.6 on two cores, where the viewer shares a core with the workers). The boards are as dense as single boards: 4.4 grid points per vertex, 65% straight traces, 7% two-bend traces, and 19–20% of the trace steps in buses, the same at every size, because every board is the same kind of problem.

Use of space. Dividing by vertices makes the grid of boards almost a tiling with equal tiles (Section 7.7). For 10^5 vertices, the 250 boards have areas of 1664 to 2088 grid points; the boards fill 94% of their grid cells, and 77% of the whole picture including the fixed gaps and frames between boards (for 25 000 vertices: 95% and 78%). Dividing the same forest by trees gives the same number of boards, but with areas from 770 to 3404 grid points: a large board widens its whole column and heightens its whole row, so the boards fill only 65% of their cells and 55% of the picture (69% and 58% for 25 000 vertices). Inside every board, the one-board algorithm keeps the density at about 4.4 grid points per vertex. Together, a forest of 400 or more vertices fills its grid cells almost completely and about three quarters of its whole drawing area, the rest being the gaps and frames that separate the boards, without any search over the arrangement of the boards.

Local repair of conflicts. When a trace cannot be placed legally, the router repairs the drawing locally, in increasing order of effort (Section 4): it retries with segments of twice the length, then uses the maze router, which may wind around obstacles; if that fails as well, it rips up up to six movable traces within distance 4 of the parent (traces to children whose own children are not placed yet), routes the new trace and then the ripped ones again, and keeps the result only if all of them fit. While a tree still has conflicts, the whole tree is laid out again from other roots and with other weights, for up to 4 seconds, and the attempt with the fewest conflicts is kept. Only what cannot be repaired in this way is drawn as a forced route, marked on the board and reported. On the test forests no route had to be forced at any size; the conflicts in Table 2 are found



(a) $p_s = 1$: thumbnails (b) $p_s = 2.6$: cached trace paths (c) $p_s = 24$: pads, rings and names

Figure 19: Levels of detail (25 000 vertices on 63 boards; the overview map at the bottom right shows the current view). p_s is the number of screen pixels per grid step.

only by the independent verification (for example a trace that passes through a pad), between 4 and 9 per 100 000 vertices from 25 000 vertices on. The exact number varies slightly between runs because the retries are bounded by time.

Level of detail. Figure 19 shows the three levels of the renderer on 25 000 vertices. Far away, every board is a cached thumbnail; at medium distance the traces are drawn from cached paths; close up, pads, gender rings, roots and names appear. Because a frame never draws more than the visible boards, the cost of a redraw does not grow with the size of the forest.

10 Discussion and limitations

What is optimised. Apart from the hard rules (clearance, exclusive trace points, no crossings, the gender rule), almost every decision is judged by how much it enlarges the board at the wanted aspect ratio: the compactness term of the cost, the squeeze, the choice among attempts, and the placement of every tree. Short traces, few bends, and buses act as secondary terms. The boards come within about 15% of the density bound of 4 grid points per vertex, so the drawing uses its area well. What is not optimised is the readability of family structure: generations are not aligned and traces may wander; this is the trade-off against the metro map.

Gender and bends. The gender rule fixes only the direction of the last segment into a vertex. Because a trace may leave its parent in any of the eight directions, a straight trace is never excluded by gender; bends are caused by obstacles, by compactness or by the bus bonus. This differs from the metro map, where the direction of a line is inherited from the parent and a gender mismatch forces a bend. Trees of two vertices are always straight (Proposition 4).

Greedy routing. Each child is routed once and kept unless rip-up or the squeeze moves it, so early decisions can block later ones. The reservation rule, the look-ahead, rip-up, multi-start and squeeze remove most dead ends for trees of up to about 100 vertices; beyond that, forced routes appear and the routing time grows like $n_T^{2.4}$ (Figure 17). Dividing a forest into boards keeps every board small, but a single huge tree is still routed on one board, since trees are never cut. Promising extensions are routing sub-trees bottom-up into precomputed shapes, or exact methods such as mixed-integer programming, which has been used for metro maps [9, 2], for small instances.

Many boards. The partition ignores the structure of the forest beyond tree sizes; families never span boards, but related families are not kept together. Because the partition by vertices makes all boards nearly equal, the grid fills its cells almost completely; the only space it adds is the fixed gap and frame around each board, a small price for bounded routing time, parallelism and fast display. The number of vertices per board

can be chosen freely: smaller boards finish sooner and are easier to read, larger boards have fewer gaps. The measured throughput depends on the number of cores; the times reported here were taken on two cores.

Degree and labels. By Proposition 2, no valid drawing exists for a vertex of degree above eight; such conflicts are reported. Pad labels are fitted inside the pads; long names would need larger pitches or external labels.

A Parameters

Parameter	Symbol	Default	Meaning
Every board			
Max. segment length	L	6	longest segment in the first enumeration stage
Pad clearance	c	2 (or 3)	minimum Chebyshev distance between pads; template trace length
Compactness	λ	1.5	weight of bounding-box growth
Bus bundling	σ	0.5	bonus per step parallel to a neighbouring trace
Board aspect	α	1.6	target width/height of a board
Two-bend traces, squeeze		on, on	doglegs and chamfered corners; squeeze pass
Tree classes		≤ 3 / 4–8 / ≥ 9	small / medium / large
Bend penalty		0.7	per 45° bend
Exit look-ahead		60	per child of v beyond $\min(\varphi'(e), \Lambda'(e))$ usable exits
Forced mode		—	fewest conflicts first, then the cost (1)
Maze router		0.7 / 3.0, 60 000	45° / 90° turn costs, expansions
Retry budget, extra attempts		4 s, 0	per tree while routes conflict; for a smaller board
One board			
Maximum vertices		about 1000	larger forests are divided into boards
Squeeze time	T_{squeeze}	800 ms	shared by the routed trees of the board
Many boards			
Maximum vertices		10^6	
Vertices per board	b	400	any value from 10 to 20 000; $k = \lceil N/b \rceil$ boards
Balance by		vertices	or trees (round-robin; boards of very different size)
Partition seed		1	reproducible shuffle
Squeeze time	T_{squeeze}	150 ms	per board
Worker threads	W	$\min(8, \text{cores} - 1)$	at least one
Detail thresholds	p_s	1.6, 4, 6, 8, 12	screen pixels per grid step (Section 7)

References

[1] J. Korst, V. Pronk and J. J. van Wijk. A visualization of family relations inspired by the London metro map. In *Proc. 13th Int. Symposium on Visual Information Communication and Interaction (VINCI 2020)*, ACM, 2020. https://doi.org/10.1145/3430036.3430065

[2] H. Bast, P. Brosi and S. Storandt. Metro maps on octilinear grid graphs. In *EuroVis 2020*.

[3] E. K. Burke, G. Kendall and G. Whitwell. A new placement heuristic for the orthogonal stock-cutting problem. *Operations Research* 52(4):655–671, 2004.

[4] E. W. Dijkstra. A note on two problems in connexion with graphs. *Numerische Mathematik* 1:269–271, 1959.

[5] R. L. Graham. Bounds on multiprocessing timing anomalies. *SIAM J. Applied Mathematics* 17(2):416–429, 1969.

[6] C. Jordan. Sur les assemblages de lignes. *J. Reine Angew. Math.* 70:185–190, 1869.

[7] C. Y. Lee. An algorithm for path connections and its applications. *IRE Trans. Electronic Computers* EC-10(3):346–365, 1961.

[8] R. Nair. A simple yet effective technique for global wiring. *IEEE Trans. Computer-Aided Design* 6(2):165–172, 1987.

[9] M. Nöllenberg and A. Wolff. Drawing and labeling high-quality metro maps by mixed-integer programming. *IEEE Trans. Visualization and Computer Graphics* 17(5):626–641, 2011.